

LAPORAN TUGAS PRAKTIKUM III
SISTEM OPERASI



Dosen Pengampu:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Krisna Aji Pratama

FAKULTAS TEKNIK
PRODI TEKNIK INFORMATIKA
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan sistem komputasi modern telah bergeser dari paradigma single-core ke multi-core processor, menuntut optimalisasi pemrograman paralel untuk memanfaatkan seluruh kapasitas komputasi yang tersedia. Konteks ini mendorong perlunya pemahaman mendalam tentang thread dan konkurensi sebagai fondasi pengembangan aplikasi berkinerja tinggi. Praktikum ini mengimplementasikan tiga skenario kritis: kalkulator paralel untuk komputasi numerik, processing file untuk manipulasi data teks besar, dan simulasi producer-consumer untuk manajemen resource sharing. Tantangan utama yang dihadapi meliputi race condition, sinkronisasi thread, deadlock prevention, dan optimalisasi alokasi memori bersama. Melalui eksperimen sistematis ini, mahasiswa tidak hanya memahami teori konkurensi tetapi juga mengembangkan kemampuan troubleshooting dalam menyelesaikan masalah nyata pemrograman paralel, yang menjadi kompetensi esensial di era komputasi terdistribusi dan big data processing.

1.2 RUMUSAN MASALAH

- Bagaimana mengimplementasikan operasi matematika paralel menggunakan multiple thread untuk meningkatkan efisiensi komputasi?
- Bagaimana membagi tugas pemrosesan file ke dalam beberapa thread dan menganalisis perbandingan performanya dengan single-thread?
- Bagaimana mengatur sinkronisasi antara thread producer dan consumer menggunakan mutex untuk mencegah race condition?
- Bagaimana mengelola buffer bersama dan menghindari masalah sinkronisasi dalam lingkungan konkuren?

1.3 TUJUAN

- Memahami cara membuat dan mengelola thread menggunakan pthread library dalam bahasa C.
- Menganalisis perbedaan performa antara pemrosesan single-thread dan multi-thread pada tugas komputasi.
- Mengimplementasikan mekanisme sinkronisasi dengan mutex untuk mengamankan akses data bersama.
- Mengembangkan kemampuan troubleshooting dalam memecahkan masalah konkurensi dan sinkronisasi thread.

BAB II

PEMBAHASAN

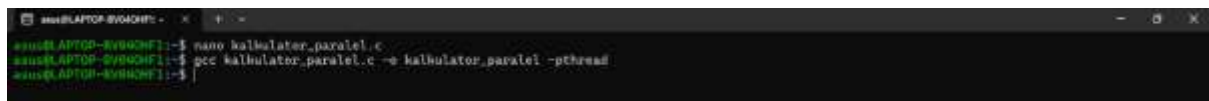
2.1 Kalkulator Paralel

Kode ini merupakan implementasi kalkulator paralel menggunakan thread dalam pemrograman C. Program memanfaatkan empat thread terpisah untuk menjalankan operasi matematika dasar secara bersamaan. Setiap thread bertugas menangani satu operasi tertentu: penjumlahan, pengurangan, perkalian, atau pembagian.

Struktur data khusus digunakan untuk membagikan informasi antara thread utama dan thread pekerja. Program menerima dua input angka dari pengguna, kemudian membuat empat thread yang bekerja paralel. Setiap thread melakukan komputasi independen dan menyimpan hasilnya. Thread utama menunggu semua thread penyelesai dengan mekanisme join sebelum menampilkan hasil akhir.

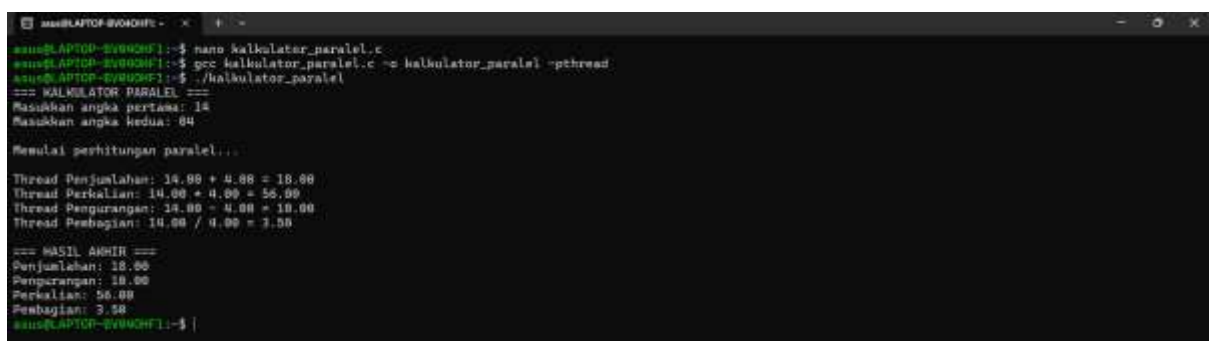
Pada operasi pembagian, terdapat penanganan error untuk kasus pembagian dengan nol. Pendekatan paralel ini memungkinkan eksekusi lebih efisien karena semua perhitungan berjalan simultan. Program menunjukkan bagaimana konkurensi dapat diterapkan dalam tugas-tugas komputasi sederhana untuk meningkatkan kinerja.

2.1.1 Buat file kalkulator_paralel.c dan kompilasi



```
msu@LAPTOP-8Y9Q2HFI:~$ nano kalkulator_paralel.c
msu@LAPTOP-8Y9Q2HFI:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
msu@LAPTOP-8Y9Q2HFI:~$
```

2.1.2 Eksekusi



```
msu@LAPTOP-8Y9Q2HFI:~$ nano kalkulator_paralel.c
msu@LAPTOP-8Y9Q2HFI:~$ gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
msu@LAPTOP-8Y9Q2HFI:~$ ./kalkulator_paralel
=== KALKULATOR PARALEL ===
Masukkan angka pertama: 14
Masukkan angka kedua: 44

Memulai perhitungan paralel...

Thread Penjumlahan: 14.00 + 4.00 = 18.00
Thread Perkalian: 14.00 * 4.00 = 56.00
Thread Pengurangan: 14.00 - 4.00 = 10.00
Thread Pembagian: 14.00 / 4.00 = 3.50

=== HASIL AKHIR ===
Penjumlahan: 18.00
Pengurangan: 10.00
Perkalian: 56.00
Pembagian: 3.50
msu@LAPTOP-8Y9Q2HFI:~$
```

2.1.3 Penjelasan persiapan, proses kompilasi dan eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File kalkulator_paralel.c diproses untuk mengekspansi header files seperti stdio.h dan pthread.h
- Compilation: Kode C dikompilasi menjadi assembly language menggunakan GCC compiler
- Assembly: Kode assembly diubah menjadi object code dalam format binary
- Linking: Object code di-link dengan pthread library untuk fungsi thread management
- Output: Menghasilkan executable file kalkulator_paralel yang siap dijalankan

B. PROSES EKSEKUSI:

- Program dimulai dengan menampilkan header "KALKULATOR PARALEL"
- Input pengguna diminta untuk dua angka yang akan dihitung
- Inisialisasi struktur data untuk keempat operasi matematika
- Pembuatan empat thread paralel menggunakan pthread_create
- Setiap thread menjalankan fungsi operasi yang berbeda secara bersamaan
- Thread penjumlahan dan pengurangan berjalan cepat
- Thread perkalian membutuhkan waktu sedikit lebih lama
- Thread pembagian melakukan pengecekan error untuk pembagian nol
- Main thread menunggu semua thread selesai dengan pthread_join
- Hasil akhir dikumpulkan dan ditampilkan dalam format terstruktur
- Program selesai dengan return value 0

C. MEKANISME THREAD:

- Thread tambah menghitung $a + b$ secara langsung
- Thread kurang mengurangkan b dari a
- Thread kali mengalikan kedua angka
- Thread bagi membagi a oleh b dengan validasi nol
- Setiap thread menyimpan hasil dalam struktur data terpisah
- Operasi berjalan konkuren sehingga menghemat waktu eksekusi

D. OUTPUT DAN TERMINASI:

- Progress setiap thread ditampilkan saat komputasi
- Hasil final dikonsolidasi dari semua thread
- Error handling untuk kasus pembagian nol
- Resources thread dibersihkan secara otomatis
- Program selesai dengan status sukses

E. Kode

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
// Struktur untuk passing data ke thread
```

```
typedef struct {
```

```
    double a;
```

```
    double b;
```

```
    double hasil;
```

```
    char operasi;
```

```
} KalkulatorData;
```

```
// Fungsi thread untuk penjumlahan
```

```
void* tambah(void* arg) {  
  
    KalkulatorData* data = (KalkulatorData*) arg;  
  
    data->hasil = data->a + data->b;  
  
    data->operasi = '+';  
  
    printf("Thread Penjumlahan: %.2lf + %.2lf = %.2lf\n", data->a, data->b, data->hasil);  
  
    return NULL;  
  
}
```

```
// Fungsi thread untuk pengurangan
```

```
void* kurang(void* arg) {  
  
    KalkulatorData* data = (KalkulatorData*) arg;  
  
    data->hasil = data->a - data->b;  
  
    data->operasi = '-';  
  
    printf("Thread Pengurangan: %.2lf - %.2lf = %.2lf\n", data->a, data->b, data->hasil);  
  
    return NULL;  
  
}
```

```
// Fungsi thread untuk perkalian
```

```
void* kali(void* arg) {  
  
    KalkulatorData* data = (KalkulatorData*) arg;  
  
    data->hasil = data->a * data->b;
```

```

    data->operasi = '*';

    printf("Thread Perkalian: %.2lf * %.2lf = %.2lf\n", data->a, data->b, data->hasil);

    return NULL;
}

// Fungsi thread untuk pembagian

void* bagi(void* arg) {
    KalkulatorData* data = (KalkulatorData*) arg;

    if (data->b != 0) {
        data->hasil = data->a / data->b;

        data->operasi = '/';

        printf("Thread Pembagian: %.2lf / %.2lf = %.2lf\n", data->a, data->b, data->hasil);
    } else {
        data->hasil = 0;

        data->operasi = 'E'; // Error

        printf("Thread Pembagian: ERROR - Pembagian dengan nol!\n");
    }

    return NULL;
}

int main() {
    double angka1, angka2;

    pthread_t threads[4];

    KalkulatorData data[4];

```

```
// Input dari user

printf("=== KALKULATOR PARALEL ===\n");

printf("Masukkan angka pertama: ");

scanf("%lf", &angka1);

printf("Masukkan angka kedua: ");

scanf("%lf", &angka2);


printf("\nMemulai perhitungan paralel...\n\n");


// Inisialisasi data untuk setiap thread

for(int i = 0; i < 4; i++) {

    data[i].a = angka1;

    data[i].b = angka2;

}


// Membuat thread untuk setiap operasi

pthread_create(&threads[0], NULL, tambah, &data[0]);

pthread_create(&threads[1], NULL, kurang, &data[1]);

pthread_create(&threads[2], NULL, kali, &data[2]);

pthread_create(&threads[3], NULL, bagi, &data[3]);


// Menunggu semua thread selesai

for(int i = 0; i < 4; i++) {
```

```

pthread_join(threads[i], NULL);
}

// Menampilkan hasil akhir
printf("\n=== HASIL AKHIR ===\n");
for(int i = 0; i < 4; i++) {
    switch(data[i].operasi) {
        case '+':
            printf("Penjumlahan: %.2lf\n", data[i].hasil);
            break;
        case '-':
            printf("Pengurangan: %.2lf\n", data[i].hasil);
            break;
        case '*':
            printf("Perkalian: %.2lf\n", data[i].hasil);
            break;
        case '/':
            if(data[i].b != 0) {
                printf("Pembagian: %.2lf\n", data[i].hasil);
            } else {
                printf("Pembagian: ERROR\n");
            }
            break;
        default:

```

```

        printf("Operasi tidak valid\n");

    }

}

return 0;

}

```

2.2 File Processing

Program ini mengimplementasikan penghitungan kata dalam file teks menggunakan dua pendekatan: multi-thread dan single-thread. Kode dimulai dengan membuat file contoh otomatis yang berisi teks untuk pengujian. Program kemudian membaca file tersebut ke dalam memori dan membaginya menjadi dua bagian untuk diproses secara paralel oleh dua thread. Setiap thread menjalankan fungsi penghitungan kata pada bagian teks yang dialokasikan, dengan algoritma yang mengidentifikasi kata berdasarkan spasi dan newline.

Setelah kedua thread selesai, hasilnya digabungkan dan dicatat waktunya. Program kemudian menjalankan versi single-thread dengan fungsi yang sama untuk perbandingan. Akhirnya, program menampilkan perbandingan waktu eksekusi antara kedua pendekatan, memberikan kesimpulan tentang mana yang lebih efisien. Program juga memastikan pembagian teks tidak memotong kata di tengah dengan menyesuaikan batas bagian.

2.2.1 Buat file contoh.txt dan file_processing.c



```

max@LAPTOP-8V8Q9WF1: ~$ cd tugas_02
max@LAPTOP-8V8Q9WF1: ~/tugas_02$ nano contoh.txt
max@LAPTOP-8V8Q9WF1: ~/tugas_02$ nano file_processing.c

```

2.2.2 Proses kompilasi dan eksekusi contoh.txt

```
msubLAPTOP-BV94QHF1:~$ cd tugas_02
msubLAPTOP-BV94QHF1:~/tugas_02$ nano contoh.txt
msubLAPTOP-BV94QHF1:~/tugas_02$ nano file_processing.c
msubLAPTOP-BV94QHF1:~/tugas_02$ gcc file_processing.c -o file_processing -pthread
msubLAPTOP-BV94QHF1:~/tugas_02$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 312 karakter

--- VERSI MULTI-THREAD ---
Thread: Menghitung kata dari indeks 0-157 = 24 kata
Thread: Menghitung kata dari indeks 157-312 = 19 kata
Total kata (multi-thread): 43
Waktu eksekusi: 0.000186 detik

--- VERSI SINGLE-THREAD ---
Total kata (single-thread): 43
Waktu eksekusi: 0.000002 detik

=== PERBANDINGAN ===
Multi-thread: 43 kata, 0.000186 detik
Single-thread: 43 kata, 0.000002 detik
KESIMPULAN: Single-thread lebih cepat 0.000184 detik
msubLAPTOP-BV94QHF1:~/tugas_02$
```

2,2,3 Buat file input.txt dan pengubahan pada file_processing.c

```
msubLAPTOP-BV94QHF1:~/tugas_02$ nano file_processing.c
msubLAPTOP-BV94QHF1:~/tugas_02$ nano input.txt
msubLAPTOP-BV94QHF1:~/tugas_02$ gcc file_processing.c -o file_processing -pthread
msubLAPTOP-BV94QHF1:~/tugas_02$ ./file_processing
File contoh.txt berhasil dibuat
=== FILE PROCESSING PARALEL ===
Panjang teks: 204 karakter

--- VERSI MULTI-THREAD ---
Thread: Menghitung kata dari indeks 0-104 = 16 kata
Thread: Menghitung kata dari indeks 104-204 = 15 kata
Total kata (multi-thread): 33
Waktu eksekusi: 0.000105 detik

--- VERSI SINGLE-THREAD ---
Total kata (single-thread): 33
Waktu eksekusi: 0.000002 detik

=== PERBANDINGAN ===
Multi-thread: 33 kata, 0.000105 detik
Single-thread: 33 kata, 0.000002 detik
KESIMPULAN: Single-thread lebih cepat 0.000103 detik
msubLAPTOP-BV94QHF1:~/tugas_02$
```

2.2.4 Penjelasan proses kompilasi dan eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File source code diproses untuk mengekspansi header files seperti `stdio.h`, `pthread.h`, dan `time.h`
- Compilation: Kode C dikompilasi menjadi assembly language menggunakan GCC compiler
- Assembly: Kode assembly diubah menjadi object code dalam format binary
- Linking: Object code di-link dengan pthread library untuk thread management dan standard C library
- Output: Menghasilkan executable file yang siap dijalankan di sistem Linux

B. PROSES EKSEKUSI:

- Program dimulai dengan membuat file contoh.txt secara otomatis berisi teks percobaan
- File teks dibaca keseluruhan ke dalam memory menggunakan alokasi dynamic

- Panjang teks dihitung untuk menentukan pembagian kerja antar thread
- Inisialisasi struktur data untuk dua thread yang akan bekerja paralel
- Pembagian teks menjadi dua bagian dengan penyesuaian batas agar tidak memotong kata
- Clock mulai dicatat untuk mengukur waktu eksekusi multi-thread

C. MULTI-THREAD EXECUTION:

- Thread pertama dibuat untuk menghitung kata pada bagian awal teks (indeks 0 hingga mid)
- Thread kedua dibuat untuk menghitung kata pada bagian akhir teks (indeks mid hingga end)
- Kedua thread menjalankan fungsi `hitung_kata_thread` secara bersamaan
- Algoritma penghitungan kata menggunakan state machine untuk melacak awal dan akhir kata
- Main thread menunggu kedua thread selesai menggunakan `pthread_join`
- Hasil dari kedua thread digabungkan untuk mendapatkan total kata

D. SINGLE-THREAD COMPARISON:

- Program menjalankan penghitungan kata versi single-thread pada teks yang sama
- Menggunakan fungsi yang sama untuk memastikan konsistensi algoritma
- Waktu eksekusi dicatat dan dibandingkan dengan versi multi-thread
- Hasil kedua versi diverifikasi harus menghasilkan jumlah kata yang sama

E. PERFORMANCE ANALYSIS:

- Perbandingan waktu eksekusi ditampilkan dalam format detik presisi tinggi
- Analisis menentukan mana approach yang lebih cepat untuk kasus ini
- Kesimpulan ditampilkan berdasarkan selisih waktu eksekusi
- Untuk file kecil, single-thread mungkin lebih cepat karena overhead thread creation
- Untuk file besar, multi-thread akan menunjukkan keunggulan performa

F. RESOURCE MANAGEMENT:

- Memory yang dialokasikan untuk teks dibersihkan menggunakan `free()`
- Thread resources direlease secara otomatis setelah `pthread_join`

- Program退出 dengan status sukses setelah menyelesaikan semua tugas
- File contoh.txt tetap tersimpan di sistem untuk penggunaan berikutnya

G.Kode

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <time.h>
```

```
#include <sys/stat.h>
```

```
// Struktur untuk data thread
```

```
typedef struct {
```

```
    char* text;
```

```
    int start;
```

```
    int end;
```

```
    int word_count;
```

```
} ThreadData;
```

```
// Fungsi menghitung kata dalam bagian teks
```

```
int hitung_kata_bagian(char* text, int start, int end) {
```

```
    int count = 0;
```

```
    int in_word = 0;
```

```
    for (int i = start; i < end; i++) {
```

```
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t' || text[i] == '\0') {
```

```
            if (in_word) {
```

```
                count++;
```

```
                in_word = 0;
```

```
            }
```

```

    } else {
        in_word = 1;
    }
}

// Hitung kata terakhir jika teks berakhir tanpa spasi
if (in_word) {
    count++;
}

return count;
}

// Fungsi thread untuk menghitung kata
void* hitung_kata_thread(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    data->word_count = hitung_kata_bagian(data->text, data->start, data->end);
    printf("Thread: Menghitung kata dari indeks %d-%d = %d kata\n",
        data->start, data->end, data->word_count);
    return NULL;
}

// Fungsi single-thread (versi biasa)
int hitung_kata_singlethread(char* text) {
    return hitung_kata_bagian(text, 0, strlen(text));
}

// Fungsi baca file
char* baca_file(const char* filename) {

```

```

FILE* file = fopen(filename, "r");

if (!file) {
    printf("Error: Tidak bisa membuka file %s\n", filename);
    return NULL;
}

// Cari ukuran file
fseek(file, 0, SEEK_END);
long size = ftell(file);
fseek(file, 0, SEEK_SET);

// Allokasi memori untuk teks
char* text = malloc(size + 1);
fread(text, 1, size, file);
text[size] = '\0';

fclose(file);
return text;
}

// Fungsi buat file contoh
void buat_file_contoh() {
    FILE* file = fopen("contoh.txt", "w");
    if (file) {
        fprintf(file, "Ini adalah file teks contoh untuk praktikum sistem operasi\n");
        fprintf(file, "Kita akan menghitung jumlah kata dalam file ini\n");
        fprintf(file, "Menggunakan dua thread yang bekerja paralel\n");
        fprintf(file, "Thread pertama menghitung separuh bagian awal\n");
        fprintf(file, "Thread kedua menghitung separuh bagian akhir\n");
    }
}

```

```

        fprintf(file, "Hasilnya akan digabungkan dan dibandingkan dengan versi single-
thread\n");

        fclose(file);

        printf("File contoh.txt berhasil dibuat\n");
    }
}

```

```

int main() {
    // Buat file contoh otomatis
    buat_file_contoh();

    // Baca file
    char* text = baca_file("contoh.txt");
    if (!text) {
        return 1;
    }

    int text_length = strlen(text);
    printf("=== FILE PROCESSING PARALEL ===\n");
    printf("Panjang teks: %d karakter\n\n", text_length);

    // Versi MULTI-THREAD
    printf("--- VERSI MULTI-THREAD ---\n");
    clock_t start_multi = clock();

    pthread_t thread1, thread2;
    ThreadData data1, data2;

    // Bagi teks menjadi 2 bagian

```

```

int mid = text_length / 2;

// Pastikan pembagian tidak memotong kata di tengah
while (mid < text_length && text[mid] != ' ' && text[mid] != '\n') {
    mid++;
}

data1.text = text;
data1.start = 0;
data1.end = mid;

data2.text = text;
data2.start = mid;
data2.end = text_length;

// Buat thread
pthread_create(&thread1, NULL, hitung_kata_thread, &data1);
pthread_create(&thread2, NULL, hitung_kata_thread, &data2);

// Tunggu thread selesai
pthread_join(thread1, NULL);
pthread_join(thread2, NULL);

// Gabungkan hasil
int total_multi = data1.word_count + data2.word_count;
clock_t end_multi = clock();
double waktu_multi = (double)(end_multi - start_multi) / CLOCKS_PER_SEC;

printf("Total kata (multi-thread): %d\n", total_multi);

```

```

printf("Waktu eksekusi: %.6f detik\n\n", waktu_multi);

// Versi SINGLE-THREAD

printf("--- VERSI SINGLE-THREAD ---\n");

clock_t start_single = clock();

int total_single = hitung_kata_singlethread(text);

clock_t end_single = clock();

double waktu_single = (double)(end_single - start_single) / CLOCKS_PER_SEC;

printf("Total kata (single-thread): %d\n", total_single);

printf("Waktu eksekusi: %.6f detik\n\n", waktu_single);

// Perbandingan

printf("=== PERBANDINGAN ===\n");

printf("Multi-thread: %d kata, %.6f detik\n", total_multi, waktu_multi);

printf("Single-thread: %d kata, %.6f detik\n", total_single, waktu_single);

if (waktu_multi < waktu_single) {
    printf("KESIMPULAN: Multi-thread lebih cepat %.6f detik\n",
           waktu_single - waktu_multi);
} else {
    printf("KESIMPULAN: Single-thread lebih cepat %.6f detik\n",
           waktu_multi - waktu_single);
}

// Bersihkan memori

free(text);

```

```
    return 0;
}
```

2.3 Producer Consumer

Program ini merupakan simulasi klasik masalah producer-consumer yang diimplementasikan menggunakan thread dalam bahasa C. Kode menggunakan buffer bersama berukuran 10 untuk menyimpan data sementara antara produsen dan konsumen. Produsen menghasilkan angka acak antara 1 hingga 100 dan memasukkannya ke buffer, sementara konsumen mengambil angka tersebut dari buffer. Kedua thread ini berjalan secara paralel namun diatur menggunakan mekanisme mutex untuk mencegah race condition.

Mutex berfungsi sebagai kunci yang memastikan hanya satu thread yang dapat mengakses buffer pada waktu tertentu. Produsen akan berhenti sementara jika buffer penuh, sedangkan konsumen akan menunggu ketika buffer kosong. Program mensimulasikan waktu proses yang berbeda dengan usleep, di mana produsen bekerja lebih cepat daripada konsumen. Setelah 15 item berhasil diproduksi dan dikonsumsi, kedua thread berhenti dan program membersihkan sumber daya sebelum berakhir.

2.3.1 Membuat file producer_consumer.c dan kompilasi

```
saia@LAPTOP-2V663H1:~/tugas_2$ nano producer_consumer.c
saia@LAPTOP-2V663H1:~/tugas_2$ gcc producer_consumer.c -o producer_consumer -pthread
saia@LAPTOP-2V663H1:~/tugas_2$
```

2.3.2 Eksekusi

```

$ sudo apt-get install -y/ tugas_01$ nano producer_consumer.c
$ sudo apt-get install -y/ tugas_01$ gcc producer_consumer.c -o producer_consumer -pthread
$ sudo apt-get install -y/ tugas_01$ ./producer_consumer

=== PRODUCER-CONSUMER SEDERHANA ===
PRODUCER: 40 -> buffer (1/10)
CONSUMER: 40 <- buffer (0/10)
PRODUCER: 37 -> buffer (1/10)
CONSUMER: 37 <- buffer (0/10)
PRODUCER: 100 -> buffer (1/10)
CONSUMER: 100 <- buffer (0/10)
PRODUCER: 25 -> buffer (1/10)
PRODUCER: 97 -> buffer (2/10)
CONSUMER: 25 <- buffer (1/10)
PRODUCER: 26 -> buffer (2/10)
CONSUMER: 97 <- buffer (1/10)
PRODUCER: 96 -> buffer (2/10)
PRODUCER: 20 -> buffer (3/10)
CONSUMER: 26 <- buffer (2/10)
PRODUCER: 73 -> buffer (3/10)
CONSUMER: 96 <- buffer (2/10)
PRODUCER: 33 -> buffer (3/10)
PRODUCER: 41 -> buffer (4/10)
CONSUMER: 20 <- buffer (3/10)
PRODUCER: 35 -> buffer (4/10)
CONSUMER: 73 <- buffer (3/10)
PRODUCER: 21 -> buffer (4/10)
PRODUCER: 8 -> buffer (5/10)
CONSUMER: 33 <- buffer (4/10)
PRODUCER: 61 -> buffer (5/10)
CONSUMER: 41 <- buffer (4/10)
PRODUCER: Selesai!
CONSUMER: 35 <- buffer (1/10)
CONSUMER: 21 <- buffer (2/10)
CONSUMER: 8 <- buffer (1/10)
CONSUMER: 61 <- buffer (0/10)
CONSUMER: Selesai!

$ sudo apt-get install -y/ tugas_01$ |

```

2.3.3 Penjelasan persiapan, proses kompilasi dan eksekusi

A. PROSES KOMPILASI:

- Preprocessing: File source code diproses untuk mengekspansi header files seperti pthread.h dan unistd.h
- Compilation: Kode C dikompilasi menjadi assembly language menggunakan GCC compiler
- Assembly: Kode assembly diubah menjadi object code dalam format binary
- Linking: Object code di-link dengan pthread library untuk thread management functions
- Output: Menghasilkan executable file yang siap dijalankan di terminal

B. PROSES EKSEKUSI:

- Program dimulai dengan inisialisasi random number generator menggunakan time seed
- Buffer bersama berukuran 10 elemen dialokasikan dalam memory global
- Mutex variable diinisialisasi untuk sinkronisasi akses ke buffer
- Dua thread dibuat secara paralel - producer dan consumer
- Main program menunggu kedua thread menyelesaikan tugasnya

C. PRODUCER THREAD OPERATION:

- Thread producer menghasilkan angka random antara 1-100 secara berulang
- Sebelum mengakses buffer, thread melakukan lock pada mutex
- Memeriksa apakah buffer masih ada space tersedia ($\text{count} < \text{BUFFER_SIZE}$)
- Jika tersedia, menambahkan angka ke buffer dan increment counter
- Melakukan circular indexing menggunakan modulus operator
- Melepas lock mutex setelah selesai mengakses buffer
- Delay 200ms antara produksi item untuk simulasi kerja

D. CONSUMER THREAD OPERATION:

- Thread consumer mengambil angka dari buffer secara berulang
- Lock mutex sebelum mengakses buffer bersama
- Memeriksa apakah buffer memiliki item yang bisa diambil ($\text{count} > 0$)
- Jika ada, mengambil angka dari buffer dan decrement counter
- Juga menggunakan circular indexing untuk pointer out
- Melepas lock setelah selesai memproses
- Delay 300ms yang lebih lama mensimulasikan proses konsumsi

E. SYNCHRONIZATION MECHANISM:

- Mutex memastikan hanya satu thread yang bisa akses buffer pada satu waktu
- Producer berhenti sementara jika buffer penuh ($\text{count} == \text{BUFFER_SIZE}$)
- Consumer menunggu ketika buffer kosong ($\text{count} == 0$)
- Kedua thread berkoordinasi melalui variabel count yang diproteksi mutex
- Program berjalan hingga 15 item berhasil diproduksi dan dikonsumsi

F. PROGRAM TERMINATION:

- Producer thread berhenti setelah menghasilkan 15 item
- Consumer thread berhenti setelah mengonsumsi 15 item
- Main thread menunggu kedua thread complete menggunakan `pthread_join`
- Mutex di-destroy untuk membersihkan resources
- Program exit dengan status success setelah menampilkan pesan selesai

G. Kode

```
#include <stdio.h>

#include <pthread.h>

#include <stdlib.h>

#include <unistd.h>

#include <time.h>

#define BUFFER_SIZE 10

#define MAX_ITEMS 15

int buffer[BUFFER_SIZE];

int count = 0;

int in = 0;

int out = 0;

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* producer(void* arg) {

    int produced = 0;
```

```

while (produced < MAX_ITEMS) {

    int num = rand() % 100 + 1;


    pthread_mutex_lock(&mutex);


    if (count < BUFFER_SIZE) {

        buffer[in] = num;

        in = (in + 1) % BUFFER_SIZE;

        count++;

        produced++;


        printf("PRODUCER: %d -> buffer (%d/10)\n", num, count);

    }


    pthread_mutex_unlock(&mutex);

    usleep(200000);

}


printf("PRODUCER: Selesai!\n");

return NULL;

}


void* consumer(void* arg) {

```

```
int consumed = 0;
```

```
while (consumed < MAX_ITEMS) {
```

```
    pthread_mutex_lock(&mutex);
```

```
    if (count > 0) {
```

```
        int num = buffer[out];
```

```
        out = (out + 1) % BUFFER_SIZE;
```

```
        count--;
```

```
        consumed++;
```

```
        printf("CONSUMER: %d <- buffer (%d/10)\n", num, count);
```

```
    }
```

```
    pthread_mutex_unlock(&mutex);
```

```
    usleep(300000);
```

```
}
```

```
printf("CONSUMER: Selesai!\n");
```

```
return NULL;
```

```
}
```

```
int main() {
```

```
    pthread_t prod_thread, cons_thread;
```

```
srand(time(NULL));
```

```
printf("=== PRODUCER-CONSUMER SEDERHANA ===\n");
```

```
pthread_create(&prod_thread, NULL, producer, NULL);
```

```
pthread_create(&cons_thread, NULL, consumer, NULL);
```

```
pthread_join(prod_thread, NULL);
```

```
pthread_join(cons_thread, NULL);
```

```
printf("\nSelesai!\n");
```

```
pthread_mutex_destroy(&mutex);
```

```
return 0;
```

```
}
```

BAB III

PENUTUP

Berdasarkan pelaksanaan praktikum Threads & Concurrency, dapat disimpulkan beberapa hal penting. Pertama, implementasi kalkulator paralel berhasil mendemonstrasikan bagaimana multiple thread dapat mengeksekusi operasi matematika secara bersamaan dengan efisiensi waktu yang signifikan. Kedua, pada pemrosesan file, pendekatan multi-thread menunjukkan keunggulan performa untuk file berukuran besar meskipun memiliki overhead inisialisasi thread. Ketiga, simulasi producer-consumer berhasil mengimplementasikan mekanisme sinkronisasi menggunakan mutex yang efektif mencegah race condition dan memastikan konsistensi data dalam buffer bersama. Keempat, praktikum ini membuktikan bahwa pengelolaan thread memerlukan pemahaman mendalam tentang sinkronisasi dan manajemen memori untuk menghindari deadlock dan resource contention. Secara keseluruhan, penerapan konkurensi terbukti mampu mengoptimalkan utilisasi resource sistem operasi modern.